```
    [mark] => 1
    [1] => VW
    [3] => GTI
    [4] => GTI
)
```

The value `1` is present in `$car1` and `$car2`, so is added to the results. The key for `1` is `'mark'` in `$car1`, `3` in `$car2`, and `$car1` keys are preserved in the results. `GTI` is found in both arrays, occurs twice in `$car1`, so we get both elements in the results (with `$car1` keys… apologies for car/key punning).

Just like `array_diff()`, `array_intersect()` will also perform type coercion, and sort the arrays, before comparison:

```
print_r(array_intersect([1, 2, 3], ['3', 2, '1']));
print_r(array_intersect(
    ['make' => 'VW', 'model' => 'Golf'],
    ['model' => 'Golf', 'make' => 'VW'],
));
```

```
Output:
Array
(
    [0] => 1
    [1] => 2
    [2] => 3
)
Array
(
    [make] => VW
    [model] => Golf
)
```

Comparing Keys

For associative arrays, it is possible to find elements with common keys between an array and one or more comparison arrays using the built-in `array_intersect_key()`.